Q-OPTIMA 2026

Finding a useful Trotter step count when quantum circuits are noisy

Qiskit Fall Fest 2026 | Phase 2 | Quantum Simulation, PS-S3

A project report by 3AM ENGINEERS

Team lead Dheeraj Koushik

Team members Sampurna Biswas, Rishi Matam, Sachin Sreenivas, Dhruv Om, Giri Samarth

In brief

The usual way to improve a Trotter approximation is to use more steps. On a noisy device, though, every extra step also means more gates. Our project looks at where those two effects balance for a small quantum simulation. We reproduced the required PS-S3 benchmark, designed a four-qubit processor layout, and built a simple advisor that estimates a useful number of steps.

The clearest result is on our custom processor: the best tested step count changes from 8 to 7 to 6 as noise increases from 0.5× to 1× to 2×. We also tested a quadratic advisor on five additional simulated noise settings. It matched the best step count in three cases and had a mean fidelity regret of 0.000422.

All results in this report come from classical quantum-circuit simulations. We have not tested the method on real quantum hardware.

1 Why we worked on this problem

In digital quantum simulation, the evolution of a system is approximated using a sequence of gates. A first-order Trotter circuit splits the Hamiltonian into parts that are easier to implement. Adding steps usually improves the approximation in the small-step limit, but it also makes the compiled circuit longer. The extra gates can introduce enough noise to cancel the gain in accuracy.

We wanted to answer a practical question: for a given noise level, how many Trotter steps should we actually use? Rather than assume that the largest circuit is best, we measured the trade-off and then tried to predict the best step count.

System and simulation settings

We used the four-qubit transverse-field Ising model specified in PS-S3, with periodic interactions between neighboring qubits. The Hamiltonian is

$H = -(Z_0Z_1 + Z_1Z_2 + Z_2Z_3 + Z_3Z_0) - (X_0 + X_1 + X_2 + X_3)$.

Both coupling constants are 1. We start from $|1010\rangle$ and evolve for total time $T = 2$. For each experiment, we apply first-order Trotterization with n steps, where n belongs to {1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 14, 16}.

Our benchmark covers Processors A, B and D at the standard noise level, plus two extra noise settings for D. This gives 60 noisy simulation results across five configurations. The base noise parameters are 0.001 for one-qubit gates, 0.005 for two-qubit gates and 0.01 for readout; the basis gates are rz, sx, x and cx.

Mathematical model: how we generate the quantum evolution

We split the Hamiltonian into the interaction part $H_{ZZ} = -(Z_0Z_1 + Z_1Z_2 + Z_2Z_3 + Z_3Z_0)$ and the transverse-field part $H_X = -(X_0 + X_1 + X_2 + X_3)$. The exact evolution is $U(T) = \exp(-iHT)$. Because these two parts do not generally commute, applying their evolutions one after the other is an approximation.

For n first-order Trotter steps, our approximation is $U_T(T,n) = [\exp(-i\, H_{ZZ}\, T/n)\, \exp(-i\, H_X\, T/n)]^n$. Each step contains the ring interactions and the transverse-field rotations. We compare the resulting state $|\psi_T(n)\rangle$ with the exact state $|\psi_{exact}\rangle$, using $F_{ideal}(n) = |\langle\psi_{exact}|\psi_T(n)\rangle|^2$. The ordering is fixed by the first-order circuit construction.

How we evaluated the circuits

We compared the ideal Trotter output with exact evolution, then compared the noisy output with the same exact target. For each processor and noise setting, we selected the step count with the highest noisy fidelity among the 12 tested values. We also recorded circuit depth, two-qubit gates, swaps and routing overhead.

An important detail is that ideal fidelity was not strictly increasing at every sampled step count. For example, it was about 0.289 at n = 2 but 0.066 at n = 3. The general accuracy-versus-depth intuition is useful, but the actual numerical sweep matters.

2 Processor layout and routing

Processor D is a four-qubit ring with connections (0,1), (1,2), (2,3) and (3,0). We chose this layout because the logical ZZ interactions in our Hamiltonian have exactly the same ring structure. The layout therefore avoids routing swaps in the tested benchmark circuits.

Processor A is a five-qubit branching tree, while Processor B is a nine-qubit heavy-hex-inspired layout. At n = 8, the compiled resources were:

Processor	Physical qubits	Logical 2Q	Compiled 2Q	Depth	Swaps
A	5	64	109	134	15
B	9	64	64	89	0
D	4	64	64	89	0

The routing overhead for A was 45 additional two-qubit gates. Both B and D had zero overhead in this comparison. D achieved the same compiled gate count and depth as B while using four rather than nine physical qubits. Their noisy fidelities at 1× were equal in this benchmark, so we do not claim that D is more accurate than B.

What happened as noise increased?

Processor	Noise scale	Best steps	Fidelity at best steps
A	1.0×	6	0.577419
B	1.0×	7	0.662816
D	0.5×	8	0.782118
D	1.0×	7	0.662816
D	2.0×	6	0.503076

The result for D shows the central effect we wanted to investigate. At lower noise, eight steps gave the highest fidelity. With more noise, the optimum moved to seven and then six steps. A deeper circuit was not automatically better.

3 Checks beyond the main benchmark

We used a Bell-state diagnostic to check how the processor behaves on an adjacent pair (0,1) and a nonadjacent pair (0,2). The ideal reference fidelity was 1 for both. With noise, the estimated fidelities were 0.963257 and 0.954224, respectively, using 4096 shots.

Qubit pair	Connection	Ideal fidelity	Noisy fidelity
(0,1)	Adjacent	1.000000	0.963257
(0,2)	Nonadjacent	1.000000	0.954224

The nonadjacent pair was slightly worse in this run, which is consistent with extra compilation and noise exposure. Because the estimates come from a finite number of shots, this comparison should not be treated as a universal ranking of the two pairs.

Noise model: what errors we actually simulated

The supplied simulation helper uses a depolarizing error channel after single-qubit gates rz, sx and x, with base probability $p_1 = 0.001$. It applies two-qubit depolarizing errors to cx, cz and rzz with base probability $p_2 = 0.005$. Measurements use a symmetric readout-flip model with probability p_readout = 0.01. The noise-scale parameter multiplies all three probabilities.

There is one important measurement detail: the main noisy-state fidelity uses a density-matrix simulation and calculates $F_{noisy} = \langle \psi_{exact} | \rho_{noisy} | \psi_{exact} \rangle$. It does not include readout noise, because that calculation has no measurement. Readout errors do apply to shot-based measurement experiments, including the Bell-state diagnostic. This distinction prevents us from attributing the fidelity crossover to readout error.

Separating approximation error from noise

We also made an error-decomposition plot to see which part of the loss grows as the circuit becomes deeper. We used three quantities: total infidelity (1 − noisy fidelity), ideal approximation error (1 − ideal Trotter fidelity), and the difference between ideal and noisy fidelity.

This last difference is a useful descriptive measure of the loss associated with the noisy simulation, not an identification of a single physical error process. At the 1× setting on D, the lowest total infidelity was approximately 0.337184 at seven steps.

Experiment outputs

The repository records the benchmark in s3_scan.csv (60 rows), s3_summary.csv (5 rows), resources.csv (36 rows) and bell.csv (4 rows). The additional error and advisor experiments are saved in error_decomposition.csv (60 rows), advisor_final_validation.csv (10 rows), v2_fixed_step_scan.csv (60 rows) and v2_fixed_step_comparison.csv (30 rows).

4 Building the Q-OPTIMA advisor

Once we had the benchmark results, we tried to predict a good step count without running a full noisy sweep for every new noise setting. We used the ideal approximation error at each n and added a simple noise penalty. The coefficient was calibrated using Processor D at 1× noise.

V1 (linear): predicted error = ideal error + α_1 × noise scale × n

V2 (quadratic): predicted error = ideal error + α_2 × noise scale × n^2

The quadratic form is an empirical model. It should not be interpreted as a physical law saying that noise always grows quadratically with the number of steps.

How we chose between the models

We first compared candidate penalties on five development noise settings: 0.25×, 0.75×, 1.5×, 2.5× and 3.0×. The linear model found the exact best step count in 2/5 cases, with mean fidelity regret 0.003098. The quadratic model achieved 3/5 and regret 0.002313. Because these results influenced our choice, we treat them as development results, not independent validation.

After choosing V2, we evaluated both linear and quadratic advisors on a different five-setting group: 0.4×, 0.9×, 1.25×, 1.8× and 2.8×. Regret is the noisy fidelity of the best tested step count minus the noisy fidelity of the step count predicted by the strategy. Lower regret is better.

Strategy	Exact matches	Mean fidelity regret
Quadratic advisor (V2)	3 / 5	0.000422
Linear advisor (V1)	2 / 5	0.001943
Always use 6 steps	2 / 5	0.009714
Always use 7 steps	1 / 5	0.010040
Always use 8 steps	1 / 5	0.024877
Always use 16 steps	0 / 5	0.187292

On these five settings, V2 reduced mean regret by approximately 78.3% relative to V1 and 95.7% relative to the strongest fixed-step baseline tested (six steps). The fixed-step comparison was added after inspecting the additional evaluation results, so we present it as a descriptive comparison rather than a separately preregistered test.

5 What we learned and what comes next

The most useful lesson from this project is that Trotter depth needs to be chosen together with the noise conditions. For our small Ising model, adding steps improved the ideal approximation in the high-step regime, but noisy fidelity peaked earlier. Matching the processor connectivity to the interaction graph also reduced routing cost without requiring a larger processor.

The advisor is promising because it gives a simple decision rule based on ideal fidelity and a calibrated noise penalty. It did better than the linear alternative on the five additional settings, but three exact matches out of five is not enough to claim a generally reliable optimizer.

Limitations

We used a single four-qubit Hamiltonian, one starting state and a fixed grid of step counts. Most advisor experiments were on Processor D, with a small number of simulated noise settings. Different circuits, compilers, noise models or hardware may give different optima. All results are from classical simulation rather than a real quantum processor.

Reproducing our work

The repository includes main.ipynb, requirements.txt, the processor definition, the benchmark and Bell scripts, the advisor evaluations, plots and CSV results. The development environment used Python 3.12.10, Qiskit 2.5.2 and Qiskit Aer 0.17.2. These differ from the organizer-preferred Qiskit 1.4.6 and Aer 0.18.0, so compatibility with that preferred environment should be checked separately.

For a complete verification, run the notebook from a fresh kernel in the repository root, confirm the output row counts and check that the figures and summary tables agree with the CSV files.

Use of AI tools

We used generative AI to help draft and debug code, plan experiments, explain results and edit documentation. The reported numerical values come from simulation scripts run during the project. We have kept the limitations and model-selection process visible so the report does not imply stronger evidence than the experiments provide.

Final note

Q-OPTIMA is a small, reproducible demonstration of noise-aware Trotter-step selection. The next useful tests would involve additional Hamiltonians, processors, noise models, random seeds and real hardware.

6 Q-OPTIMA Ultimate: hybrid step and processor advisor

After evaluating the V1 and V2 predictors, we added a hybrid decision layer. It combines our best-tested step-count model (V2) with a separate processor-resource comparison. The goal is to return a practical circuit configuration without claiming a new fidelity-accuracy improvement.

How the hybrid makes a recommendation

First, choose a step count. For each candidate n, the V2 model scores the ideal Trotter error plus an empirical quadratic noise penalty: **E_pred(n,s) = [1 - F_ideal(n)] + alpha2 * s * n^2**. The coefficient is fitted using Processor D at standard (1x) noise. The step count with the smallest score is selected.

Second, choose a processor. At that step count, the advisor compares the executable compiled circuits for A, B and D. It ranks them by fewer two-qubit gates, then lower depth, then fewer physical qubits. This is a **resource-based** ranking; it is not a direct prediction of hardware fidelity.

Hybrid output and consistency check

Noise scale	Recommended steps	Selected processor	Compiled 2Q gates
0.4x, 0.5x	8	D	64
0.9x, 1.0x, 1.25x	7	D	56
1.8x, 2.0x, 2.8x	6	D	48

On five previously evaluated noise settings (0.4x, 0.9x, 1.25x, 1.8x and 2.8x), the hybrid matched the V2 step recommendation in **5/5 cases**. This checks integration consistency, **not** accuracy against the true optimum. V2 itself matched the measured optimum in **3/5 cases**, with mean fidelity regret **0.000422**.

What did not work as well

We also tested an alternative predictor using compiled gate-count information to estimate noise loss. On the same five evaluation settings it achieved **1/5** exact matches and mean fidelity regret **0.003604**. We therefore retained V2 for step selection instead of claiming that the alternative was better. This comparison uses previously examined settings, not a fresh independent test.

Scope and reproducibility

Processor D was selected at all eight displayed scales because it tied B on compiled two-qubit gates and depth for the relevant circuits while requiring four rather than nine physical qubits. This does not prove D is universally optimal. The model is calibrated on one four-qubit Hamiltonian and simulated noise conditions; its behavior on other workloads or real devices remains untested. Run **main.ipynb** and **ultimate_hybrid_advisor.py** from the repository root; the resulting table is saved to **results/tables/ultimate_hybrid_recommendations.csv**.